

# CNNs, RNNs, LSTMs & Motivation for Attention

**Niloy Ganguly**  
**Professor**  
**IIT Kharagpur**



## 01 Loss function

Measures how wrong predictions are. Training minimises this.

MSE

Cross-entropy

$$L = -\sum y_i \log(\hat{y}_i)$$

## 02 Backpropagation

Chain rule flows error backward through layers to update weights.

chain rule

gradient descent

$$\partial L / \partial w = \partial L / \partial \hat{y} \cdot \partial \hat{y} / \partial w$$

## 04 Confusion matrix

TP, TN, FP, FN counts — basis for all classifier metrics.

$$\text{Precision} = \text{TP} / (\text{TP} + \text{FP})$$

$$\text{Recall} = \text{TP} / (\text{TP} + \text{FN})$$

F1 score

Accuracy

# Parameters vs Hyperparameters

## Parameters

- Learned automatically during training
- Internal to the model
- Updated by the optimizer (e.g. gradient descent)

### Examples

- Weights & biases in a neural network
- Coefficients in linear regression

**Set by: Optimizer • Not chosen by you**

VS

## Hyperparameters

- Set before training begins
- External configuration of the model
- Chosen & tuned by the data scientist

### Examples

- Learning rate, batch size, epochs
- Number of layers, dropout rate

**Set by: You • Chosen before training**

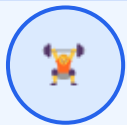
# Train / Validation / Test Sets

*Why splitting your data correctly matters*

Training Set 80%

Val 10%

Test 10%



## Training Set

- Model learns weights & patterns
- Largest split (~60–80%)
- Directly used in backpropagation
- More data → better generalization

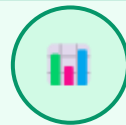
⚠ Memorizing train data = overfitting



## Validation Set

- Tune hyperparameters & architecture
- Monitor val loss to stop early
- Guides model selection decisions
- Not used in gradient updates

⚠ Leaking val labels biases selection



## Test Set

- Final unbiased performance estimate
- Used ONLY once — at the very end
- Never drives any design choice
- Reports real-world generalisation

⚠ Peeking at test = data leakage!

**Key Rule:** Each split must remain independent. Never let information from validation or test sets influence training decisions.



# Underfitting, Good Fit & Overfitting

## Underfitting

*Model is too simple — fails to capture data patterns.*

- High training error
- High test/validation error
- Model ignores key trends

Cause: Too simple model / too little training

Fix: More layers, more features, train longer

## Good Fit

*Model learns patterns well and generalizes to new data.*

- Low training error
- Low test/validation error
- Balanced complexity

Cause: Right model + right training data

Fix: This is the goal!

## Overfitting

*Model memorizes training data — fails on unseen data.*

- Very low training error
- High test/validation error
- Fits noise, not signal

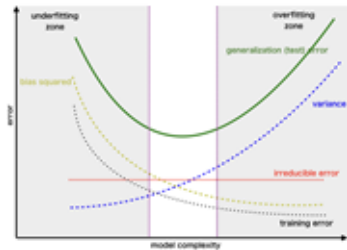
Cause: Too complex model / too little data

Fix: Regularization, dropout, more data

# Overfitting, Underfitting & Regularization

## Bias-Variance Tradeoff

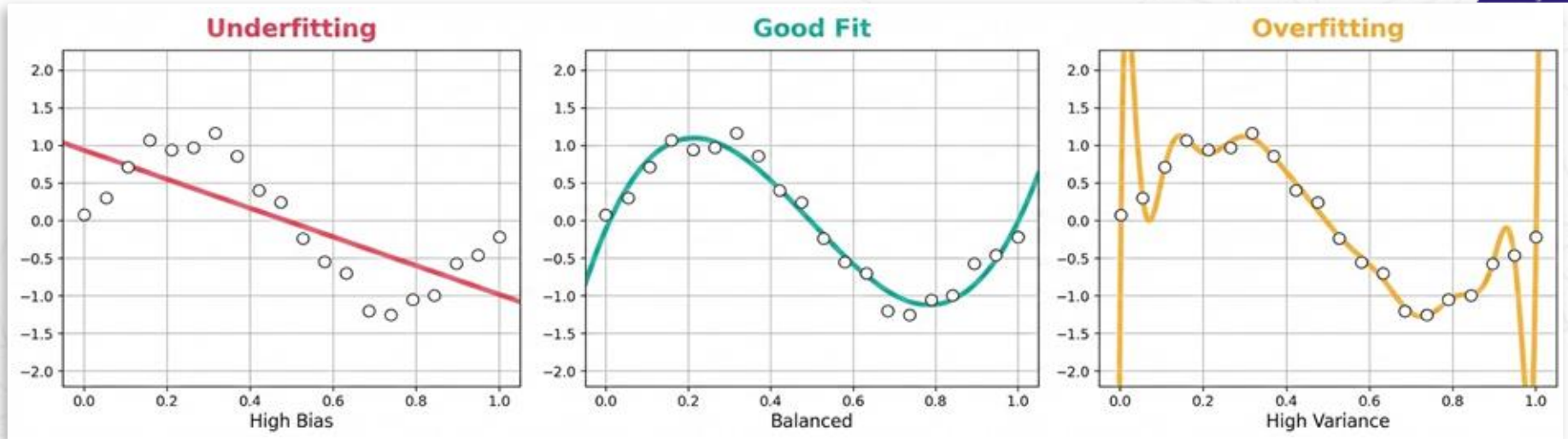
- Underfitting (High Bias): Model too simple, cannot capture patterns
- Overfitting (High Variance): Model memorizes training data, fails on new data
- The sweet spot: Generalizes well to unseen data
- Monitored via Train/Validation/Test split
- Training accuracy vs Validation accuracy gap = overfitting signal



## Regularization Techniques

- L2 Regularization (Weight Decay): Penalize large weights in loss function
- L1 Regularization: Induces sparsity (feature selection)
- Dropout: Randomly zero out neurons during training (prevents co-adaptation)
- Batch Normalization: Normalize layer activations for stable training
- Data Augmentation: Expand training set with transforms (flip, crop, noise)
- Early Stopping: Stop training when validation loss starts increasing

# What Are Overfitting & Underfitting?



## Underfitting

Model too simple — misses patterns in training data. High bias, low variance. Degree-1 poly on curved data.

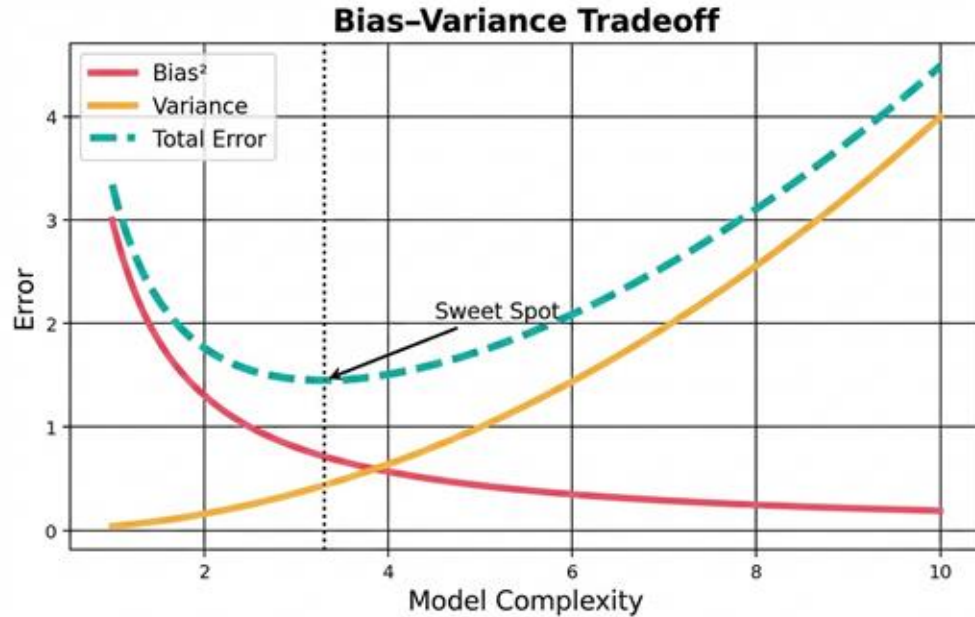
## Good Fit

Captures the true pattern without memorising noise. Low bias + low variance. The goal of every model.

## Overfitting

Model too complex — memorises noise. Low train error, high val error. High variance, low bias.

# The Bias–Variance Tradeoff



## High Bias

Underfitting. Model assumptions too rigid. Increases training error.

## High Variance

Overfitting. Model too sensitive to training data. Val error diverges.

## Sweet Spot

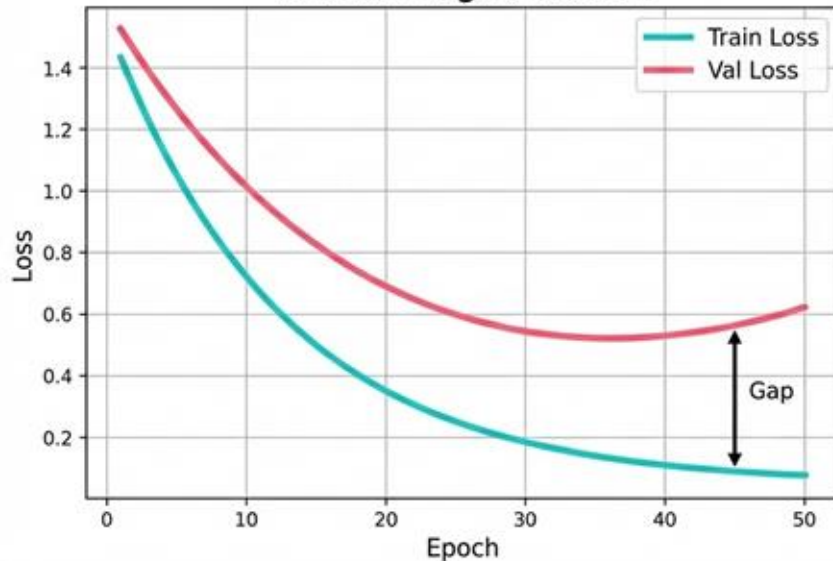
Minimise total error =  $\text{Bias}^2 + \text{Variance} + \text{irreducible noise}$ .

$$\text{Total Error} = \text{Bias}^2 + \text{Variance} + \epsilon$$

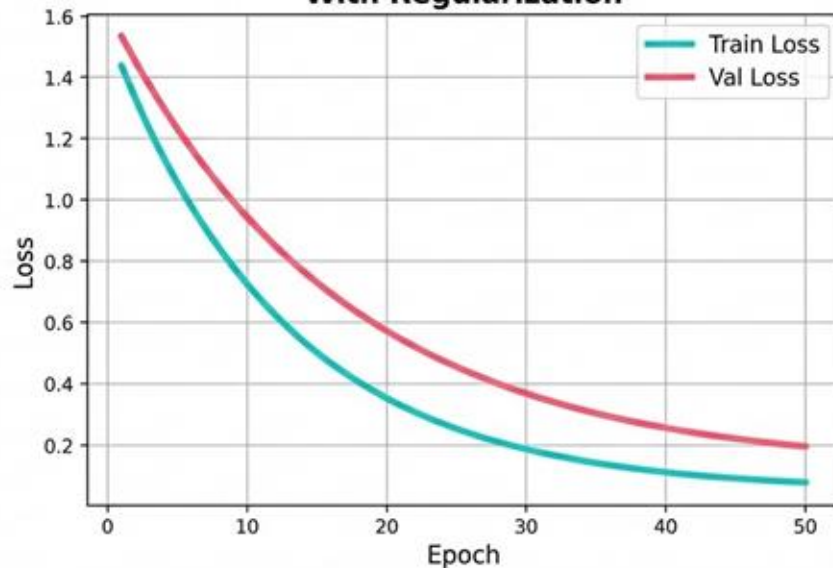


# Detecting Overfitting — Training vs Validation Loss

**Without Regularization**



**With Regularization**



● Train loss ↓ but val loss ↑ → overfitting. Stop training early or add regularization.

● With regularization, val loss tracks train loss — the gap shrinks significantly.

● Both losses staying high → underfitting. Increase model capacity or train longer.

# Regularization — Penalise Complexity

## What is Regularization?

Adds a penalty term to the loss function to discourage large weights.  
Forces the model to stay simple and generalize better.

### L2 (Ridge)

$\text{Loss} + \lambda \cdot \sum w^2$  Shrinks all weights smoothly toward zero.

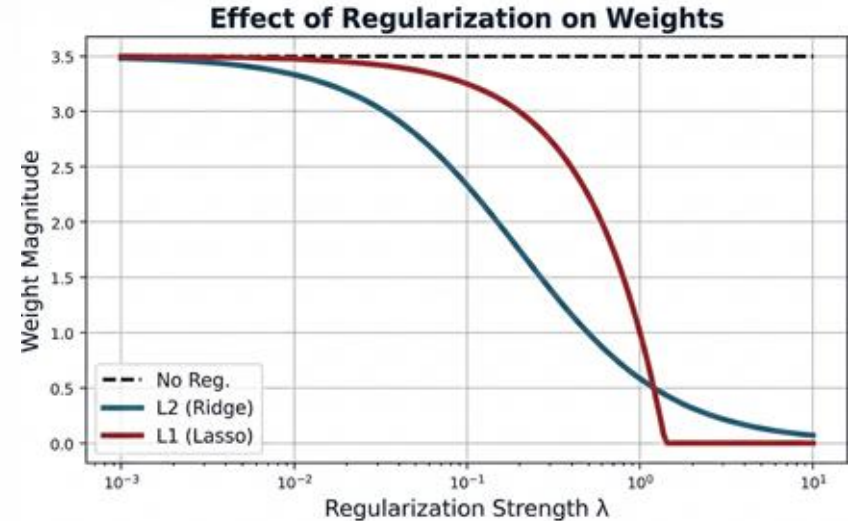
### L1 (Lasso)

$\text{Loss} + \lambda \cdot \sum |w|$  Drives some weights exactly to zero (feature selection).

```
# Sklearn: Ridge (L2) & Lasso (L1)
from sklearn.linear_model import Ridge, Lasso
```

```
ridge = Ridge(alpha=1.0) # alpha = λ
ridge.fit(X_train, y_train)
```

```
lasso = Lasso(alpha=0.1)
lasso.fit(X_train, y_train)
```





# L1 vs L2 — Geometry & Sparsity

## L1 — Lasso

*Constraint: diamond shape*

- Penalty:  $\lambda \cdot \sum |w_i|$
- Creates sparse models (weights  $\rightarrow 0$ )
- Good for feature selection
- Non-differentiable at 0
- Solver: coordinate descent

```
# L1 pushes weights to zero
lasso = Lasso(alpha=0.5)
print(lasso.coef_)
#  $\rightarrow [0. \ 1.23 \ 0. \ 0. \ 0.87]$ 
```

## L2 — Ridge

*Constraint: circle shape*

- Penalty:  $\lambda \cdot \sum w_i^2$
- Shrinks all weights uniformly
- No exact zeros — keeps all features
- Differentiable everywhere
- Has analytical closed-form solution

```
# L2 shrinks weights evenly
ridge = Ridge(alpha=1.0)
print(ridge.coef_)
#  $\rightarrow [0.12 \ 0.88 \ 0.03 \ 0.21 \ 0.75]$ 
```

# More Regularization Techniques — Dropout & Early Stopping



## Dropout

Randomly zero-out neurons during training (rate  $p$ ). Forces network to learn redundant representations and prevents co-adaptation.

```
import torch.nn as nn

model = nn.Sequential(
    nn.Linear(128, 64),
    nn.ReLU(),
    nn.Dropout(p=0.5), # 50% dropout
    nn.Linear(64, 10)
)
```

## Early Stopping

Monitor validation loss. Stop training when it stops improving (patience window). Keeps the best checkpoint automatically.

```
from tensorflow.keras.callbacks import EarlyStopping

cb = EarlyStopping(
    monitor='val_loss',
    patience=5,      # wait 5 epochs
    restore_best_weights=True
)

model.fit(X, y, callbacks=[cb])
```

# Putting It Together — Sklearn Pipeline

```
#import packages
import numpy as np
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import (
    PolynomialFeatures, StandardScaler)
from sklearn.linear_model import Ridge
from sklearn.model_selection import (
    train_test_split, cross_val_score)

np.random.seed(42)
X = np.sort(np.random.rand(80,1), axis=0)
y = np.sin(2*np.pi*X).ravel() + np.random.normal(0, 0.15, 80)
X_train, X_test, y_train, y_test = train_test_split(X, y,
    test_size=0.2)
```

```
# Regularized polynomial pipeline
pipe = Pipeline([
    ('poly', PolynomialFeatures(degree=9)),
    ('scaler', StandardScaler()),
    ('model', Ridge(alpha=1.0)) # L2
])
pipe.fit(X_train, y_train)

train_r2 = pipe.score(X_train, y_train)
test_r2 = pipe.score(X_test, y_test)
print(f"Train R2: {train_r2:.3f}")
print(f"Test R2: {test_r2:.3f}")

# Cross-validation
cv = cross_val_score(pipe, X, y, cv=5)
print(f"CV: {cv.mean():.3f} ± {cv.std():.3f}")
```

✓ Always hold out a test set — never tune on it.

✓ Use cross-validation to get stable performance estimates.

✓ Tune  $\alpha$  (lambda) with GridSearchCV or validation curve.

# Quiz 1

A model achieves 98% training accuracy but only 61% validation accuracy. Which action is MOST likely to help WITHOUT adding more training data?

- A)** Increase the number of layers and neurons
- B)** Apply L2 regularization (weight decay)
- C)** Train for more epochs to reach convergence
- D)** Remove the validation set and retrain on all data



menti.com  
2226 2447

Participant counter

# Quiz 1

A model achieves 98% training accuracy but only 61% validation accuracy. Which action is MOST likely to help WITHOUT adding more training data?

- A) Increase the number of layers and neurons
- B) Apply L2 regularization (weight decay) ☒
- C) Train for more epochs to reach convergence
- D) Remove the validation set and retrain on all data



menti.com  
2226 2447

Participant counter



## Summary — Quick Reference

| Problem              | Symptom                   | Fix                                       | Code Lever                                   |
|----------------------|---------------------------|-------------------------------------------|----------------------------------------------|
| <b>Underfitting</b>  | High train + val error    | Bigger model, more features, more epochs  | <code>degree=↑, layers=↑, epochs=↑</code>    |
| <b>Overfitting</b>   | Low train, high val error | Regularize, dropout, more data            | <code>Ridge/Lasso alpha=↑, Dropout(p)</code> |
| <b>High Variance</b> | Train >> Val score        | L2 penalty, cross-validate                | <code>Ridge(alpha), cross_val_score</code>   |
| <b>High Bias</b>     | Both scores low           | Add polynomial features, tune model class | <code>PolynomialFeatures(degree=↑)</code>    |





# The Bias-Variance Tradeoff

## High Bias = Underfitting

Model assumptions are too rigid.  
Cannot capture patterns → high error on both train and test.  
Like fitting a straight line through a curved dataset.

## High Variance = Overfitting

Model is too sensitive to training data.  
Captures noise as if it were signal → low train error, high test error.  
Like a complex curve that passes through every point.

## The Sweet Spot — Good Fit

Balance bias and variance by tuning hyperparameters, regularizing the model, and validating carefully on held-out data. Monitored via the gap between Training Loss and Validation Loss.

# Detecting Overfitting — Train vs Validation Loss



**Train loss ↓ but Val loss ↑**

Classic overfitting signal. Model memorizes training data and fails to generalize. → Stop early or add regularization.

**Both losses staying high**

Classic underfitting signal. Model lacks capacity to learn the task. → Increase model complexity or train longer.

## Key Remedies

### Regularization

L1 / L2 penalize large weights

### Dropout

Randomly zero neurons during training

### Early Stopping

Stop when val loss starts rising

### More Data

Reduces variance significantly

### Cross-Validation

Robust estimate of true performance



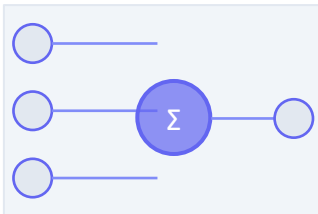
# Quick Reference Summary

| Concept         | What it is                                       | Who sets it      | Goal                        |
|-----------------|--------------------------------------------------|------------------|-----------------------------|
| Parameters      | Weights & biases — learned during training       | Optimizer (auto) | Minimize loss function      |
| Hyperparameters | Learning rate, layers, epochs, batch size, etc.  | User             | Best model architecture     |
| Underfitting    | Model too simple — misses patterns in data       | —                | Add complexity / train more |
| Good Fit        | Right balance of complexity & generalization     | —                | This is the goal ✓          |
| Overfitting     | Model memorizes training data, fails on new data | —                | Regularize / simplify       |

# From Perceptron to Specialized Architectures

1958

## Perceptron

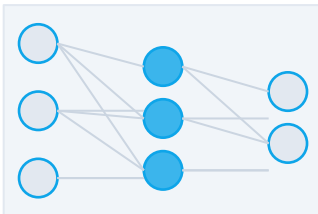


### Single neuron

- Binary output
- Linear boundary
- Step activation

1980s

## MLP



### Hidden layers

- Backpropagation
- Non-linear fn
- Universal approx

1990s

## CNN

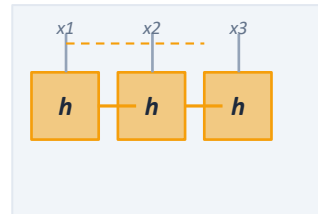


### Conv filters

- Spatial hierarchy
- Weight sharing
- Image/vision tasks

1990s

## RNN / LSTM

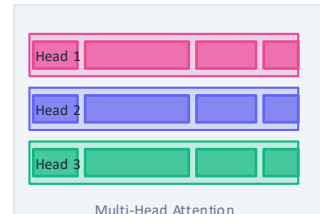


### Sequential data

- Hidden state
- Long-term memory
- NLP / time-series

2017–now

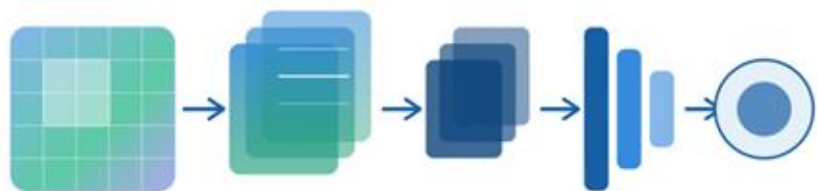
## Transformer



Multi-Head Attention

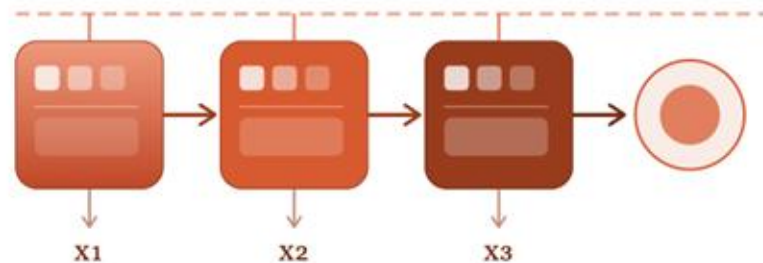
### Self-attention

- Parallelizable
- Positional enc.
- LLMs / GPT / BERT



# CNN

Convolutional Neural Network



# RNN / LSTM

Recurrent Neural Network & Long Short-Term Memory



# Convolutional Neural Networks (CNNs)

## Definition

A Convolutional Neural Network (CNN) is a specialized deep learning architecture designed to process grid-structured data — primarily images.

Inspired by the visual cortex, CNNs use learnable filters that slide over input data to detect spatial patterns like edges, textures, and shapes.

They achieve translation invariance, meaning a learned feature can be detected regardless of where it appears in the image.

*Originally proposed by LeCun et al. (1989) for digit recognition.*

## Local Connectivity



Neurons connect to small regions, not the entire input

## Shared Weights



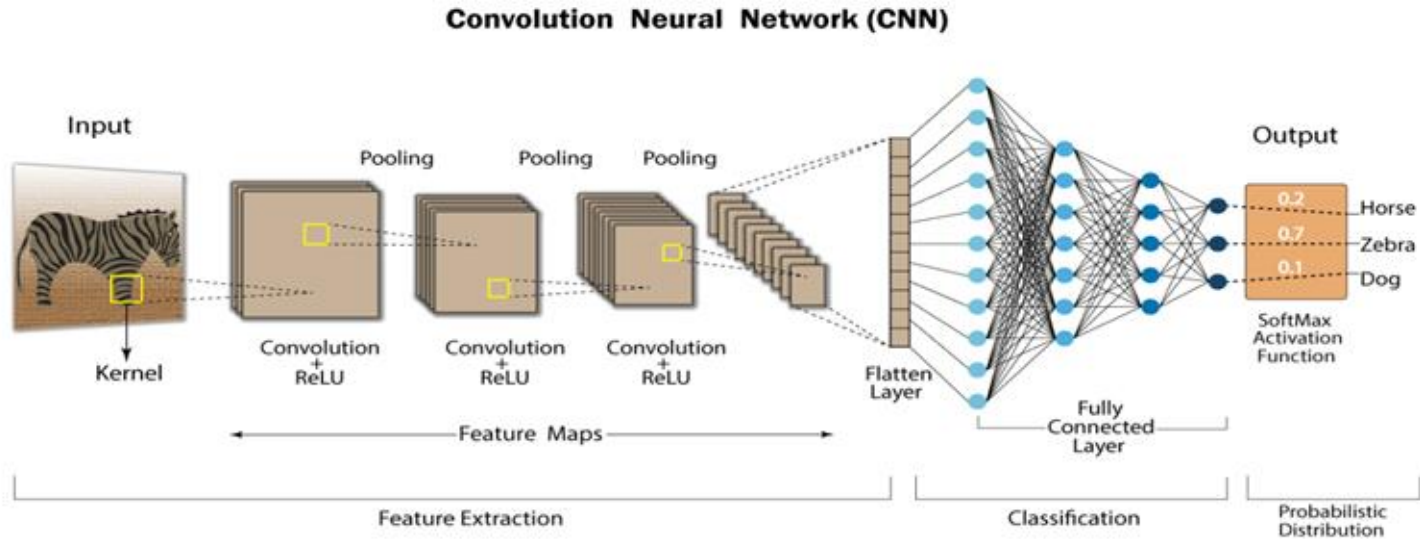
Same filter applied across all spatial locations

## Spatial Hierarchy



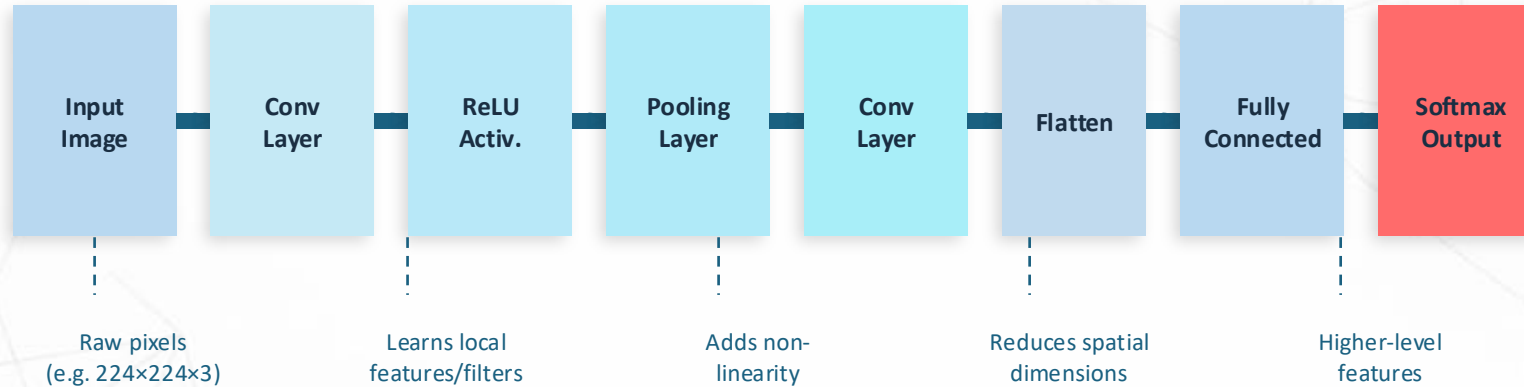
Low-level → high-level features through stacked layers

# Convolutional Neural Networks (CNNs)



- Convolution: Slide a small filter (kernel) over image to detect local features (edges, textures, patterns)
- Parameter sharing: Same filter weights reused across all spatial locations → fewer parameters
- Pooling: Downsample feature maps (MaxPool, AvgPool) → spatial invariance
- Stacking layers: Low-level (edges) → Mid-level (shapes) → High-level (object parts) → Class
- AlexNet (2012) — revolutionized image recognition; ResNet, VGG, EfficientNet followed

# CNN Architecture



*Feature Map Depth Increases → Spatial Size Decreases →*

## Early Layers

Detect edges & simple patterns

## Mid Layers

Textures, shapes, object parts

## Deep Layers

High-level semantic concepts



# Key CNN Components

## ⌘ Convolutional Layer

- Applies learnable filters (kernels) via dot product
- Produces feature maps capturing spatial patterns
- Parameters: kernel size, stride, padding, # filters

## ⌘ Pooling Layer

- Reduces spatial dimensions (downsampling)
- Max pooling takes the maximum value in region
- Provides translation invariance, reduces parameters

## ⚡ Activation (ReLU)

- Introduces non-linearity:  $f(x) = \max(0, x)$
- Prevents vanishing gradient problem
- Variants: Leaky ReLU, ELU, GELU

## © Fully Connected Layer

- Combines all spatial features into a vector
- Performs final classification/regression
- Often uses Dropout for regularization

# Convolution: Matrix Operation

Input image (5×5)

|   |   |   |   |   |
|---|---|---|---|---|
| 1 | 2 | 3 | 0 | 1 |
| 0 | 1 | 2 | 3 | 0 |
| 1 | 3 | 2 | 1 | 2 |
| 2 | 0 | 1 | 3 | 1 |

Filter / kernel (3×3)

|    |   |   |
|----|---|---|
| -1 | 0 | 1 |
| -2 | 0 | 2 |
| -1 | 0 | 1 |

×



Feature map (3×3)

|   |   |    |
|---|---|----|
| 7 | 0 | -6 |
| 3 | 1 | -2 |
| 1 | 4 | -2 |

Dot product — highlighted patch × kernel:

$$(1 \times -1) + (2 \times 0) + (3 \times 1) + (0 \times -2) + (1 \times 0) + (2 \times 2) + (1 \times -1) + (3 \times 0) + (2 \times 1) = 7$$

The filter slides across every 3×3 patch (stride = 1). Each position produces one value via element-wise multiply & sum → builds the full feature map.

# Pooling Layer — Matrix Example

Input feature map (4×4)

|    |    |    |    |
|----|----|----|----|
| 12 | 20 | 8  | 4  |
| 18 | 36 | 14 | 6  |
| 5  | 11 | 22 | 30 |
| 2  | 7  | 17 | 25 |

Max Pool  
(2×2, stride 2)



Max-pooled output (2×2)

|    |    |
|----|----|
| 36 | 14 |
| 11 | 30 |

## Window colour key + How Max Pooling works:

TL → max = 36

BL → max = 11

TR → max = 14

BR → max = 30

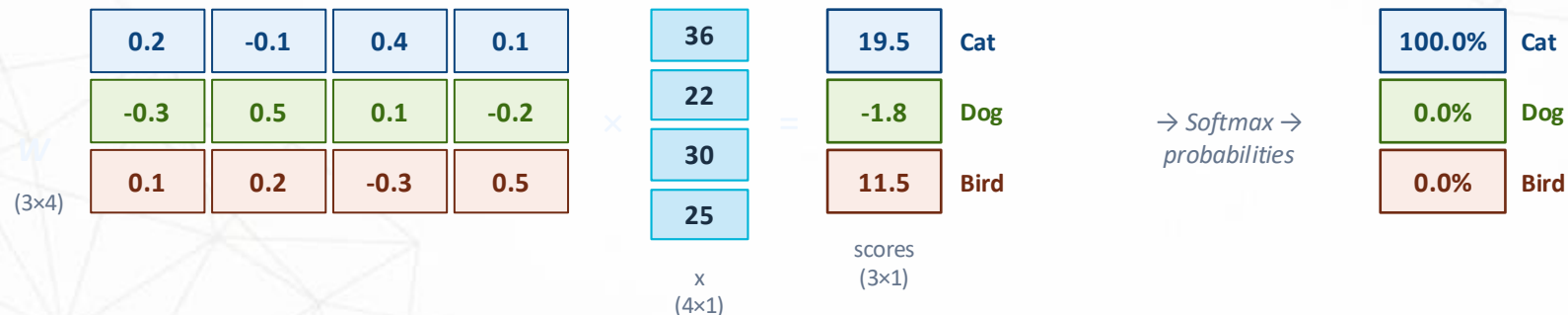
Each 2×2 window slides with stride 2 — max value per window fills one output cell → 4×4 becomes 2×2.

# Fully Connected Layer — Matrix Example

Step 1 — Flatten pooled feature map into a vector



Step 2 — Multiply by weight matrix  $W$  (3x4) → scores for 3 classes



## How the Fully Connected layer works:

The flattened vector  $x$  is multiplied by weight matrix  $W$ :  $\text{score} = W \cdot x + b$ . Each row of  $W$  learns which combination of features predicts a class. Softmax then converts raw scores into probabilities that sum to 1.

# Scikit learn → PyTorch

*Why we make the move — and what changes*

## Fixed vs. Flexible Pipelines

sklearn wraps algorithms in rigid fit/predict APIs. PyTorch lets you define any forward pass — custom loops, dynamic graphs, research ideas.

## Shallow vs. Deep Models

sklearn excels at classical ML (SVMs, trees, linear models). Deep networks — CNNs, RNNs, Transformers — need PyTorch's layer-level control.

## CPU-only vs. GPU Acceleration

sklearn runs on CPU. PyTorch moves tensors to GPU with `.to('cuda')`, enabling 10–100× speedups on large datasets and models.

## Manual vs. Autograd

In sklearn, gradients don't exist. PyTorch's autograd engine computes them automatically — essential for training any neural network.

## Black-box vs. Full Transparency

sklearn hides internals. PyTorch exposes every weight, activation, and loss — you control and inspect everything for research & debugging.

## Local Tools vs. Industry Standard

PyTorch powers research at Meta, Google, Tesla & top universities. Moving to PyTorch means speaking the language of modern deep learning.

**Bottom line:** sklearn is a great starting point. But deep learning at scale demands the flexibility, power, and ecosystem that only PyTorch delivers.

# PyTorch

## *Deep Learning in Python*



### Neural Network Building Blocks

`torch.nn.Module`, `Linear`, `Conv2d`, `RNN/LSTM/GRU`, `Transformer` — composable layers with autograd support

### Autograd & Optimization

Automatic differentiation via `torch.autograd` · `SGD`, `Adam`, `AdamW`, `RMSProp` · Learning rate schedulers

### Data Loading & Preprocessing

`Dataset`, `DataLoader`, `torchvision.transforms` · `Batching`, `shuffling`, `augmentation` · Custom Dataset API

### Key Design Principles

Dynamic computation graph · Pythonic & debuggable · NumPy-compatible tensors · GPU/CPU seamless switching

### Model Training & Evaluation

Training loops, loss functions (`CrossEntropy`, `MSE`, `NLL`) · Checkpointing · `TorchScript` & `ONNX` export

### Ecosystem & Use Cases

`torchvision`, `torchaudio`, `torchtext` · Hugging Face integration · Used in CV, NLP, RL, generative AI



# CNN in PyTorch

```
import torch          # Core PyTorch library
import torch.nn as nn  # Neural network module
```

```
class SimpleCNN(nn.Module):          # Define a simple CNN model
```

```
    def __init__(self):
        super().__init__()
```

```
    # First convolution layer:
    self.conv1 = nn.Conv2d(1, 16, kernel_size=3, padding=1) #Input: 1 channel (grayscale), Output: 16 feature
maps and kernel size = 3x3, padding=1 keeps spatial size same
```

```
    # Second convolution layer
```

```
    self.conv2 = nn.Conv2d(16, 32, kernel_size=3, padding=1) # Input: 16 channels → Output: 32 feature maps
```

```
    # MaxPooling layer
```

```
    self.pool = nn.MaxPool2d(2, 2) # Reduces spatial dimensions by half (e.g., 28x28 → 14x14)
```

```
    # Activation function
```

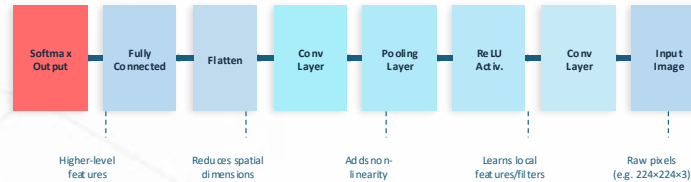
```
    self.relu = nn.ReLU()
```

```
    # Fully connected layer:
```

```
    # After two poolings: 28 → 14 → 7
```

```
    # Final feature map size = 32 × 7 × 7
```

```
    self.fc = nn.Linear(32 * 7 * 7, 10) # 10 output classes
```



**nn.Conv2d(in, out, k)**

Learnable 2D filters. Params: in/out channels, kernel\_size, padding.

**nn.MaxPool2d(2, 2)**

Takes max over 2×2 windows — halves H and W.

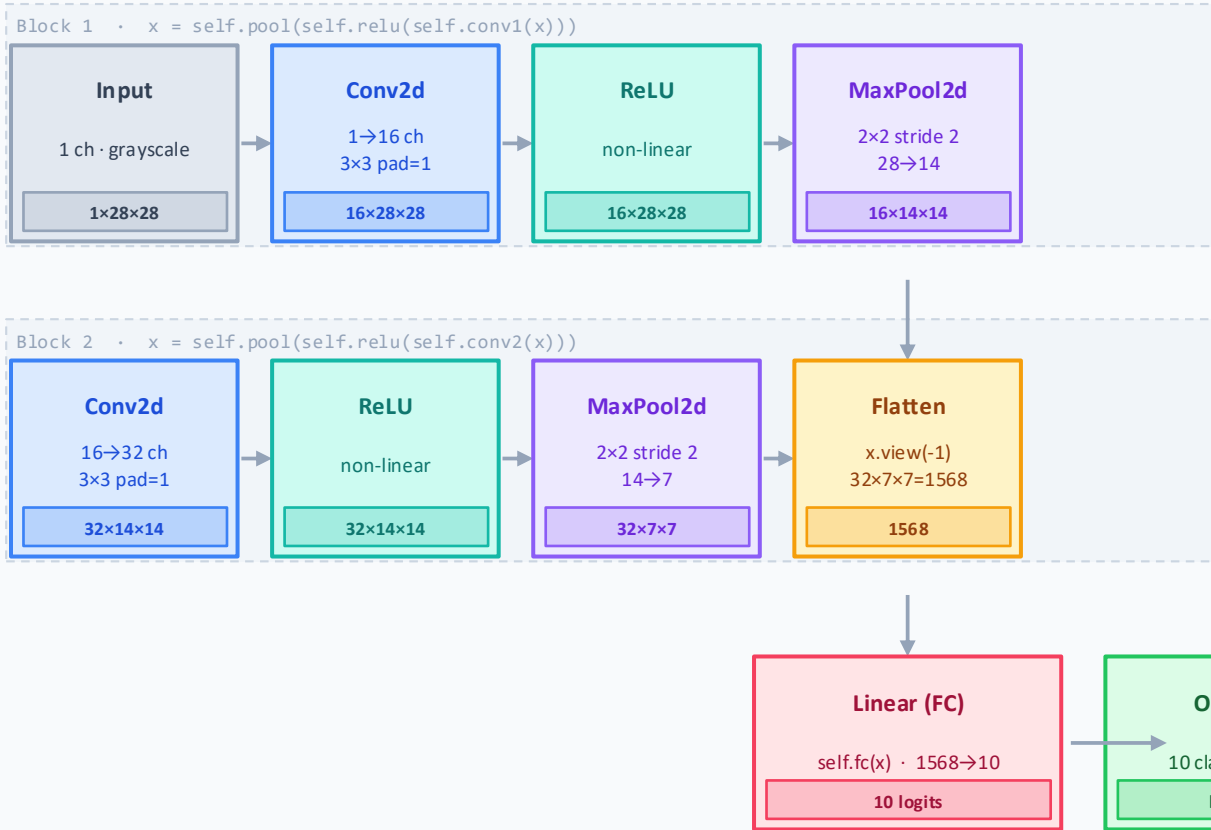
**nn.ReLU()**

$f(x) = \max(0, x)$ . Applied after every conv layer.

**nn.Linear(in, out)**

Flattened features → class scores.

# SimpleCNN — Forward Pass



```
forward(self, x)

# Block 1
x = self.pool(
    self.relu(
        self.conv1(x)))

# Block 2
x = self.pool(
    self.relu(
        self.conv2(x)))

# Flatten
x = x.view(
    x.size(0), -1)

# Output
return self.fc(x)
```



# CNN Applications & Famous Architectures

## Computer Vision

- Image Classification
- Object Detection (YOLO)
- Semantic Segmentation
- Facial Recognition

## Medical Imaging

- Tumor Detection in MRI
- Diabetic Retinopathy
- Histopathology Analysis
- COVID-19 CT Screening

## Other Domains

- NLP (Text-CNN)
- Audio Classification
- Video Understanding
- Autonomous Driving

## Landmark CNN Architectures



## Quiz 2

Consider a document containing 1000 words. The word at position 1000 needs information that appeared at position 1 to be interpreted correctly. A standard CNN can theoretically capture such information if it is made sufficiently deep. However, what practical challenge remains?

- A) CNNs cannot process sequences.
- B) Information must travel through many layers, making long-range dependencies harder to learn and represent.
- C) CNNs cannot be trained using gradient descent.
- D) CNNs require one-hot encodings.



menti.com  
2226 2447

Participant counter

## Quiz 2

Consider a document containing 1000 words. The word at position 1000 needs information that appeared at position 1 to be interpreted correctly. A standard CNN can theoretically capture such information if it is made sufficiently deep. However, what practical challenge remains?

- A) CNNs cannot process sequences.
- B) Information must travel through many layers, making long-range dependencies harder to learn and represent. 
- C) CNNs cannot be trained using gradient descent.
- D) CNNs require one-hot encodings.



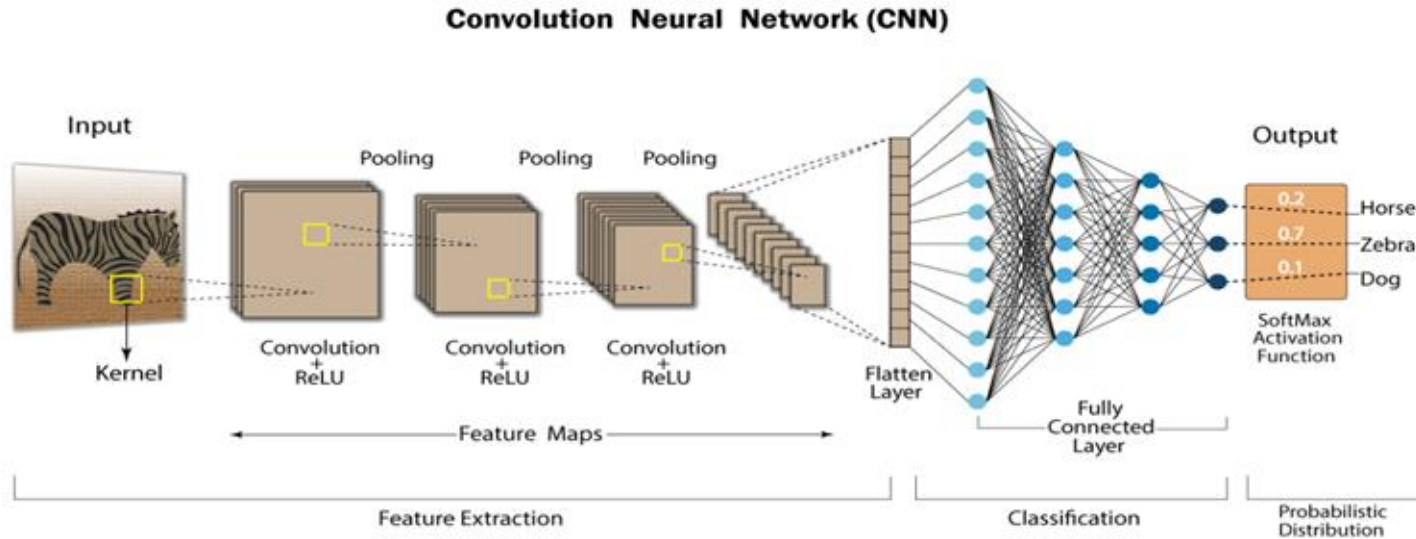
menti.com  
2226 2447

Participant counter

# Drawbacks of CNN

- CNNs are designed to process fixed-size input images and generate fixed-size outputs. They struggle with variable-length sequences, such as sentences in natural language processing (NLP) or varying-length videos.
- CNNs process data points independently or within a small local window. They cannot inherently capture the temporal relationship or context across time steps, making them inefficient for tasks where order matters, like stock forecasting or text generation.

# Convolutional Neural Networks (CNNs)

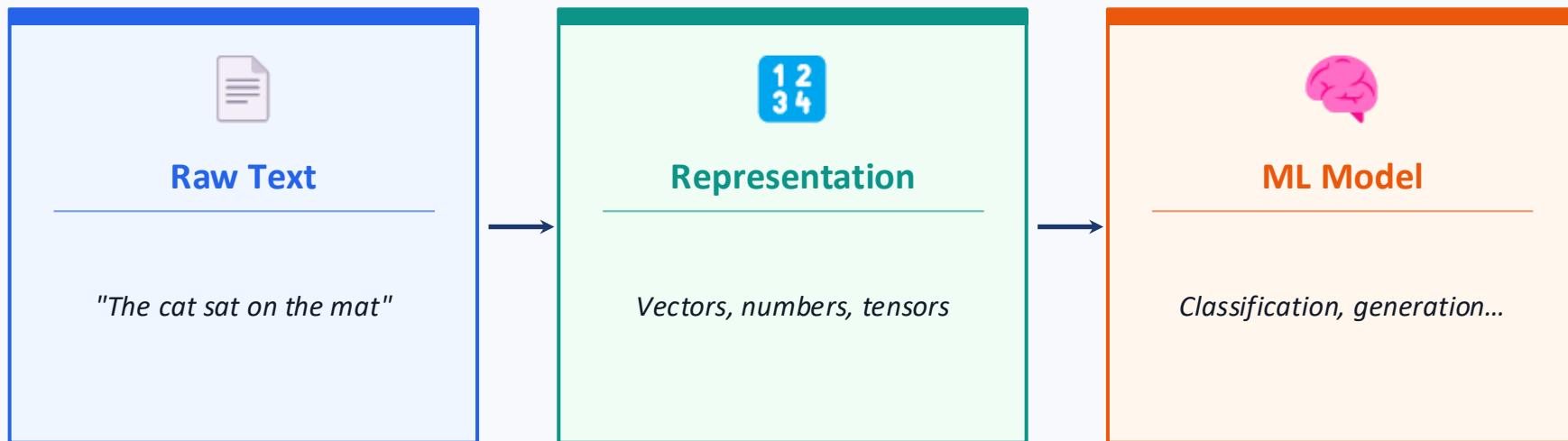


- Convolution: Slide a small filter (kernel) over image to detect local features (edges, textures, patterns)
- Parameter sharing: Same filter weights reused across all spatial locations → fewer parameters
- Pooling: Downsample feature maps (MaxPool, AvgPool) → spatial invariance
- Stacking layers: Low-level (edges) → Mid-level (shapes) → High-level (object parts) → Class
- AlexNet (2012) — revolutionized image recognition; ResNet, VGG, EfficientNet followed

# Feature Learning through Embeddings

*A motivating journey: from hand-crafted statistics to learned dense vectors*

*How do we turn raw text into something a machine can learn from?*

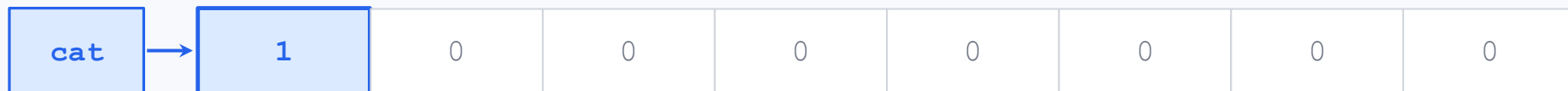


**The representation step is the key bottleneck — and the biggest opportunity.**

# One-Hot Vector Representation

Step 1: Assign every word a unique index in a fixed vocabulary **Vocabulary ( $|V| = 8$ )**

|     |     |      |       |       |      |       |     |
|-----|-----|------|-------|-------|------|-------|-----|
| 0   | 1   | 2    | 3     | 4     | 5    | 6     | 7   |
| cat | dog | king | queen | apple | runs | happy | the |

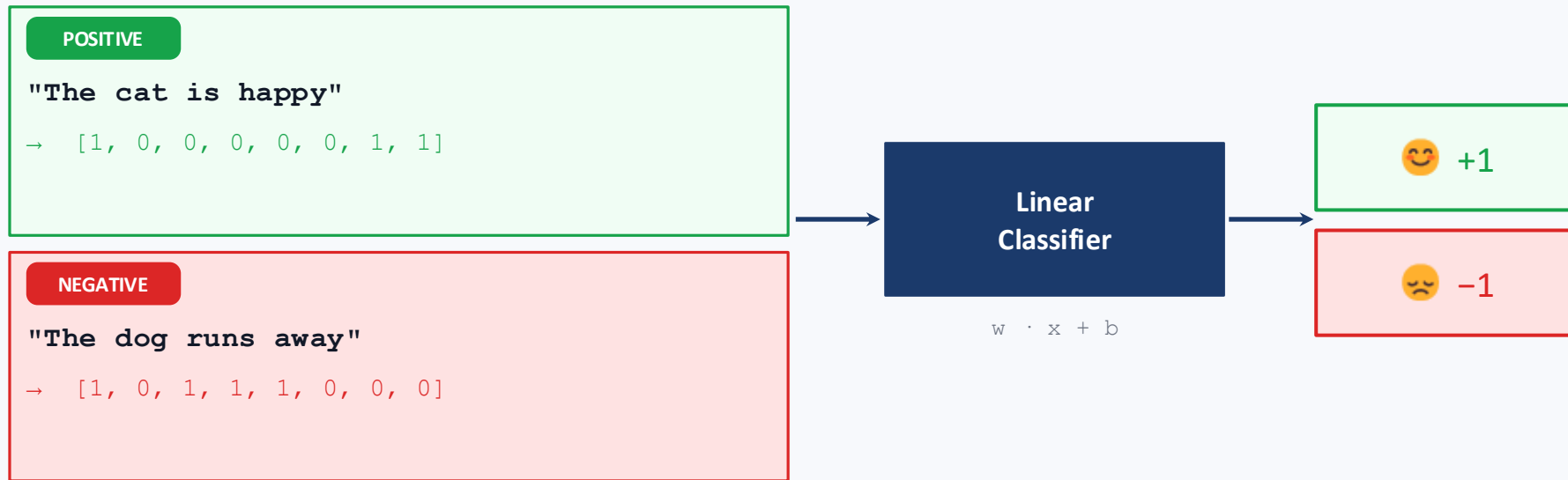


**Key property:** Each word is a point in high-dimensional space. The '1' is at index  $i$ , everything else is 0.

# Classification with One-Hot Vectors

Example: sentiment classification — can a linear model separate positive from negative?

## Bag-of-Words Representation



### ✓ What works

- Simple, fast, interpretable weights
- Works for word-frequency signals

### ⚠ What struggles

- "happy" and "joyful" treated as unrelated
- Sparse vectors → huge, mostly-zero inputs



# One-Hot Vectors

Where every NLP pipeline once began



# One-Hot Encoding

## Python

```
from sklearn.preprocessing import OneHotEncoder
import numpy as np

vocab = ["king", "queen", "man", "woman", "cat"]
# Vocab size = 50,000 in production!

encoder = OneHotEncoder(sparse_output=False)
vectors = encoder.fit_transform([[w] for w in vocab])

print(vectors.shape) # (5, 5) → (50000, 50000) in prod
# king    = [1, 0, 0, 0, 0]
# queen   = [0, 1, 0, 0, 0]
# man     = [0, 0, 1, 0, 0]

# Cosine similarity?
from numpy.linalg import norm
cos = lambda a,b: np.dot(a,b)/(norm(a)*norm(b))
print(cos(vectors[0], vectors[1])) # 0.0 ← king ≠ queen??
```



## Sparse & Huge

50K vocab → 50K-dim vector. 2.5GB matrix.  
Most values are 0.



## No Semantics

$\cos(\text{king}, \text{queen}) = 0.0$ . The model has no idea they're related.



## No Relationships

$\text{king} - \text{man} + \text{woman} = ???$  The math is meaningless.

→ We need dense, meaningful representations.

## Quiz 3

A search engine learns that users who search for "car" often click the same pages as users who search for "automobile". As an engineer, what will be your decision choice.

- **A)** One-hot representations of the words and use cosine similarity.
- **B)** Simply add "automobile" as a hardcoded synonym for "car" in a lookup table.
- **C)** Reduce one-hot vectors using PCA so that "car" and "automobile" end up closer in the reduced space.
- **D)** None of them.



menti.com  
2226 2447

Participant counter

## Quiz 3

A search engine learns that users who search for "car" often click the same pages as users who search for "automobile". As an engineer, what will be your decision choice.

- **A)** One-hot representations of the words and use cosine similarity.
- **B)** Simply add "automobile" as a hardcoded synonym for "car" in a lookup table. ☒
- **C)** Reduce one-hot vectors using PCA so that "car" and "automobile" end up closer in the reduced space.
- **D)** None of them.



menti.com  
2226 2447

Participant counter

## Quiz 4

A search engine learns that users who search for "car" often click the same pages as users who search for "automobile". As an engineer, what will be your decision choice. What can be a generalized design choice. (restaurant, eatery)

- **A)** One-hot representations of the words and use cosine similarity.
- **B)** Simply add "automobile" as a hardcoded synonym for "car" in a lookup table.
- **C)** Reduce one-hot vectors using PCA so that "car" and "automobile" end up closer in the reduced space.
- **D)** None of them.



menti.com  
2226 2447

Participant counter

## Quiz 4

A search engine learns that users who search for "car" often click the same pages as users who search for "automobile". As an engineer, what will be your decision choice. What can be a generalized design choice.

- **A)** One-hot representations of the words and use cosine similarity.
- **B)** Simply add "automobile" as a hardcoded synonym for "car" in a lookup table.
- **C)** Reduce one-hot vectors using PCA so that "car" and "automobile" end up closer in the reduced space.
- **D)** None of them. 



menti.com  
2226 2447

Participant counter

# The Search Engine Synonym Problem

 **Scenario:** A search engine observes that users who search "**car**" often click the same pages as users who search "**automobile**".

 **Observation:** Despite identical intent, the system returns different results — it treats the two words as completely unrelated.

 **Question:** As an engineer, which approach do you choose to fix this?

- A One-hot representations + cosine similarity
- B Hardcoded synonym table (car  $\leftrightarrow$  automobile)
- C PCA on one-hot vectors
- D None of the above

# Option A — One-Hot Vectors + Cosine Similarity

## Representation

**car**  $\rightarrow [1, 0, 0, 0, 0, \dots]$

**automobile**  $\rightarrow [0, 1, 0, 0, 0, \dots]$

**banana**  $\rightarrow [0, 0, 1, 0, 0, \dots]$

## What Goes Wrong?

$\cos(\text{car}, \text{automobile}) = 0$   
 $\cos(\text{car}, \text{banana}) = 0$

**✗ Verdict**

"car" and "automobile" are no closer than "car" and "banana". Click behavior is completely ignored.

Answer A is **✗ Wrong**



# Option C — PCA on One-Hot Vectors

*What PCA actually does with 6 one-hot vectors in 6D:*

1

**Center the data** Subtract mean  $\mu = [1/6, 1/6, 1/6, 1/6, 1/6, 1/6]$  from every vector.

2

**Covariance has special form**  $\Sigma = (1/6)I - (1/36)J \rightarrow$  one eigenvalue = 0, five identical eigenvalues remain.

3

**Intrinsic dimension = 5 (not 6)** All points lie in a 5D hyperplane. PCA discovers this — but nothing semantic.

4

**Words remain equidistant** In 2D projection, every word sits equally far from every other. "car"  $\neq$  closer to "automobile".

Answer C is  Wrong — PCA on one-hot vectors discovers geometry, not meaning.

# Option B — Hardcoded Synonym Table

## ✓ Pros

- Fixes the observed problem immediately
- Easy to implement and deploy
- Widely used in commercial search engines
- No training data required

## ✗ Cons

- Does not scale — manual maintenance
- Misses context-dependent synonyms
- Cannot discover new relationships automatically
- Brittle — one engineer's judgment

### ⚠ Context matters!

In a practical engineering interview: **B is a reasonable pragmatic answer.**

In an ML/representation-learning exam: **D (dense embeddings) is the intended answer.**

# Option D

## Core Idea

*Words that lead to similar user behavior should have similar vector representations.*

### ✗ One-Hot Space

car  $\rightarrow [1, 0, 0, 0, 0, \dots]$

automobile  $\rightarrow [0, 1, 0, 0, 0, \dots]$

cos similarity = 0

### ✓ Dense Embedding Space

car  $\rightarrow [0.21, -0.37, 0.88, \dots]$

automobile  $\rightarrow [0.19, -0.35, 0.85, \dots]$

cos similarity  $\approx 0.99$  

## How to learn these embeddings:



Word2Vec / GloVe on text corpora



Query-document click graphs



Neural retrieval models (DPR, BERT)

# Limitations of Sparse Representations

## No Semantic Meaning

"king" and "queen" are two unrelated dimensions — dot product = 0

"cat"  $\neq$   
"kitten" in TF-IDF

## Curse of Dimensionality

Vocabulary of 100k words = 100k-dim vectors with 99%+ zeros

99%+ entries  
are zero

## No Word Order

"dog bites man" and "man bites dog" have identical representations

Bag-of-words  
assumption

## Out-of-Vocabulary

New words at test time have no vector — model is completely blind

Zero vector =  
no info

→ These are fundamental flaws. We need a smarter representation.

# Why Sparsity Breaks Similarity

The dot product between any two one-hot vectors is always zero

**Cosine Similarity between one-hot vectors:**

|     |   |   |   |   |   |   |   |   |
|-----|---|---|---|---|---|---|---|---|
| cat | 1 | 0 | 0 | 0 | 0 | 0 | 0 | 0 |
|-----|---|---|---|---|---|---|---|---|

 • 

|        |   |   |   |   |   |   |   |   |
|--------|---|---|---|---|---|---|---|---|
| kitten | 0 | 0 | 0 | 0 | 1 | 0 | 0 | 0 |
|--------|---|---|---|---|---|---|---|---|

 = 0.00 *synonyms!*

|      |   |   |   |   |   |   |   |   |
|------|---|---|---|---|---|---|---|---|
| king | 0 | 0 | 1 | 0 | 0 | 0 | 0 | 0 |
|------|---|---|---|---|---|---|---|---|

 • 

|       |   |   |   |   |   |   |   |   |
|-------|---|---|---|---|---|---|---|---|
| queen | 0 | 0 | 0 | 1 | 0 | 0 | 0 | 0 |
|-------|---|---|---|---|---|---|---|---|

 = 0.00 *antonyms?*

|       |   |   |   |   |   |   |   |   |
|-------|---|---|---|---|---|---|---|---|
| happy | 0 | 0 | 0 | 0 | 0 | 0 | 1 | 0 |
|-------|---|---|---|---|---|---|---|---|

 • 

|     |   |   |   |   |   |   |   |   |
|-----|---|---|---|---|---|---|---|---|
| dog | 0 | 1 | 0 | 0 | 0 | 0 | 0 | 0 |
|-----|---|---|---|---|---|---|---|---|

 = 0.00 *unrelated!*



## Insight

One-hot vectors are orthogonal by construction — the model can never discover that "cat" and "kitten" are related. We need a representation where similar words are geometrically close.

# Distributional Semantics — Words by Context

"You shall know a word by the company it keeps" — J. R. Firth, 1957

## The Big Idea

- ① **A word's meaning** is defined by the words that frequently appear around it.
- ② **Context window**: look at  $\pm k$  words on either side of the target word.
- ③ **Build a vector** from the distribution of context words across a large corpus.
- ④ **Similar contexts**  $\rightarrow$  similar meaning  $\rightarrow$  similar vectors.

## Context Window ( $k = 2$ )

...government debt problems turning **banking** crises as happened in 2009...

...saying that Europe needs unified **banking** regulation to replace the hodgepodge...

...India has just given its **banking** a shot in the arm...



These context words will represent **banking**

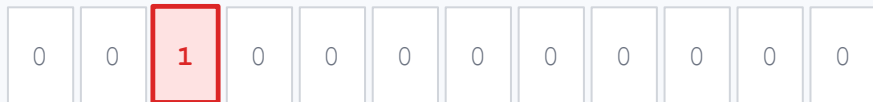
 **Key shift:** Instead of a word's position in a fixed vocabulary, we encode what words it co-occurs with.

*Count co-occurrences  $\rightarrow$  compress  $\rightarrow$  dense vector. This is the road to word embeddings.*

# Dense Representations — Distributed Semantics

Replace one sparse index with a short, dense vector where every dimension carries meaning

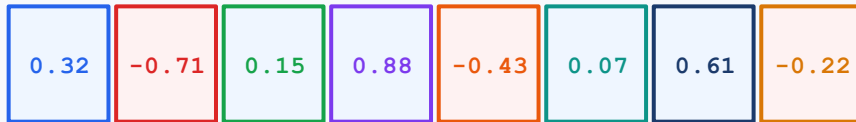
## SPARSE (One-Hot)



... (50,000 dims)

- X 99%+ zeros
- X No similarity
- X Huge memory

## DENSE (Word Embedding)



(300 dims — every value carries meaning)

learn  
weights →

- ✓ All values active
- ✓ Geometry = meaning
- ✓ Compact & fast

### Geometry encodes meaning — famous examples:

```
vec("king") - vec("man") + vec("woman") ≈ vec("queen")
```

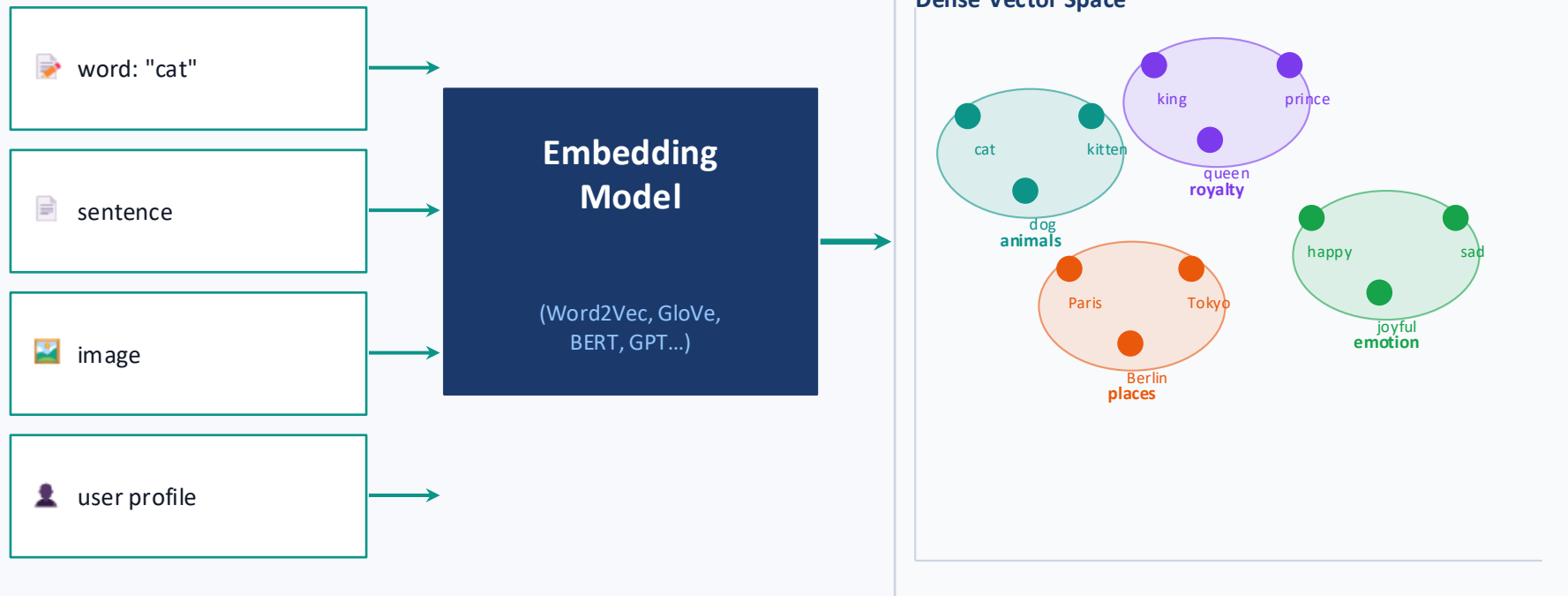
```
vec("Paris") - vec("France") + vec("Germany") ≈ vec("Berlin")
```

```
cosine( vec("cat"), vec("kitten") ) ≈ 0.92 ✓
```

# Feature Learning through Embeddings

*The embedding model learns dense vectors from raw data — no hand-crafting required*

**Any Objects**



**Embeddings !!** — Dense, low-dimensional vectors where geometry encodes linguistic meaning.



# Dense Embeddings

Compact, learned vector representations



# Embedding Lookup Tables

## PyTorch

```
import torch
import torch.nn as nn

VOCAB_SIZE = 50_000
EMBED_DIM = 300 # was 50K, now just 300!

# This is a learnable weight matrix: (50000, 300)
embedding = nn.Embedding(VOCAB_SIZE, EMBED_DIM)

# Forward pass: integer index → dense vector
token_ids = torch.tensor([42, 1337, 7]) # "king", "queen", "man"
vectors = embedding(token_ids) # shape: (3, 300)

# During training, gradients flow back and update weights
# Similar words end up with similar vectors – automatically!
optimizer = torch.optim.Adam(embedding.parameters())

print(vectors.shape) # torch.Size([3, 300])
print(f"Memory: {VOCAB_SIZE * EMBED_DIM * 4 / 1e6:.1f} MB")
# Memory: 60.0 MB vs 10 GB for one-hot!
```

## What changed?

50K dims



300 dims

✓ 167× smaller

All zeros



Dense floats

✓ Information-rich

Fixed



Learned

✓ Task-aware

# Word2Vec

Skip-gram & CBOW — learning from context



# Word2Vec: Two Training Strategies

## Skip-gram

Input: center word → Predict: context words

"The [cat] sat on the mat"



Predict: "The", "sat", "on"

Better for rare words

## CBOW

Input: context words → Predict: center word

"The [?] sat on the mat"



Context: "The", "sat", "on"

Faster, good for frequent words

## Gensim

```
from gensim.models import Word2Vec
```

```
sentences = [["king", "rules", "the", "kingdom"],  
              ["queen", "rules", "the", "kingdom"],  
              ["man", "is", "a", "human"], ["woman", "is", "a", "human"]]
```

```
model = Word2Vec(sentences, vector_size=100,  
                  window=3, min_count=1,  
                  sg=1) # sg=1: Skip-gram,  
                        sg=0: CBOW  
  
# The famous analogy  
result = model.wv.most_similar(  
    positive=["king", "woman"], negative=["man"])  
print(result[0]) # ('queen', 0.9991...)
```

## Quiz 5



A fintech startup trains a Word2Vec model on 10 years of **Bloomberg** financial articles. The model learns strong representations for terms like *equity*, *bond*, and *bank*. They then deploy it as a feature extractor for a general-purpose chatbot. A user types:

*"My grandfather used to fish by the bank every Sunday morning."*

**The model produces misleading answer**

Which of the following best explains why the model produce a misleading representation for the word *bank* in this sentence?

- A) The model was trained on too little data — with more Bloomberg articles, it would eventually learn the river-bank sense as well.
- B) Word2Vec's 300-dimensional vectors are too small to store multiple meanings simultaneously.
- C) Word2Vec collapses all usages of "bank" into a single vector during training, so at inference time it has no mechanism to consult surrounding words and shift meaning.
- D) The CBOW training objective only looks at surrounding words during training, not at inference — so context-dependent disambiguation was never part of the design.



menti.com  
2226 2447

Participant counter



## Quiz 5

A fintech startup trains a Word2Vec model on 10 years of **Bloomberg** financial articles. The model learns strong representations for terms like *equity*, *bond*, and *bank*. They then deploy it as a feature extractor for a general-purpose chatbot. A user types:

*"My grandfather used to fish by the bank every Sunday morning."*

Which of the following best explains why the model will produce a misleading representation for the word *bank* in this sentence?

- A) The model was trained on too little data — with more Bloomberg articles, it would eventually learn the river-bank sense as well.
- B) Word2Vec's 300-dimensional vectors are too small to store multiple meanings simultaneously.
- C) Word2Vec collapses all usages of "bank" into a single vector during training, so at inference time it has no mechanism to consult surrounding words and shift meaning. 
- D) The CBOW training objective only looks at surrounding words during training, not at inference — so context-dependent disambiguation was never part of the design.



menti.com  
2226 2447

Participant counter

# Static Embeddings Can't Handle Polysemy

## PyTorch

```
# Word2Vec gives ONE vector per word, regardless of context
model = Word2Vec.load("word2vec-google-news-300")

bank_vector = model.wv["bank"] # One single vector
# But "bank" means DIFFERENT things:
# "I deposited money at the bank" → financial institution
# "She sat on the river bank" → geographic feature
# "The plane had to bank left" → aeronautical maneuver

# Word2Vec collapses all meanings into ONE point in space!
print(model.wv.most_similar("bank"))
# [('banks', 0.92), ('banking', 0.89), ('financial', 0.85), ...]
# No context. No disambiguation. Just one averaged vector.
```

## "bank" in Word2Vec

**bank**  
**(1 vector)**



finance?



river?



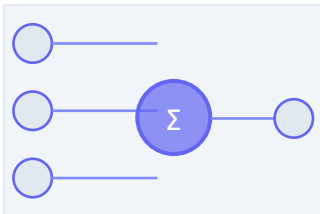
plane?

💡 Fix: Embeddings should be computed dynamically based on the surrounding context.

# From Perceptron to Specialized Architectures

1958

## Perceptron

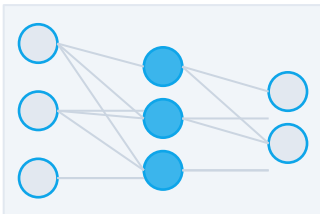


### Single neuron

- Binary output
- Linear boundary
- Step activation

1980s

## MLP



### Hidden layers

- Backpropagation
- Non-linear fn
- Universal approx

1990s

## CNN

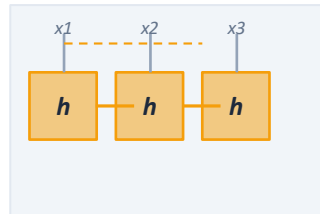


### Conv filters

- Spatial hierarchy
- Weight sharing
- Image/vision tasks

1990s

## RNN / LSTM

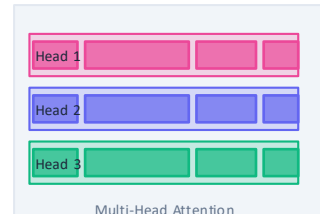


### Sequential data

- Hidden state
- Long-term memory
- NLP / time-series

2017–now

## Transformer



### Self-attention

- Parallelizable
- Positional enc.
- LLMs / GPT / BERT



# Thank you

## Instructor



**Niloy Ganguly**  
niloy@cse.iitkgp.ac.in

## Teaching Assistants (IITKGP)



**Sombit Bose**



**Shrimon Mukherjee**